

Module 5 - Software Assurance & Secure SQL

Ryan Gogerty - JHU Modern Concepts in Python, Module 5

1. Installation and Running (pip and uv)

The project ships with both a pip/venv path and a uv path so a brand-new environment can reproduce it exactly. With pip: create a virtual environment (`python3 -m venv .venv`), activate it, run `'pip install -r requirements.txt'`, then `'pip install -e .'` to install the package in editable mode. With uv: run `'uv venv'` to create the environment, activate it, then `'uv pip sync requirements.txt'` followed by `'uv pip install -e .'`. The key difference is that `'uv pip sync'` makes the environment match the requirements file exactly - it removes any package not listed - which guarantees a clean, reproducible build every time. The editable install (`-e .`) registers `src/` as a real Python package so imports resolve identically in local runs, in pytest, and in CI, eliminating the fragile `sys.path` manipulation used previously. To run the app, set the `DB_*` environment variables (or `DATABASE_URL`) and run `'python src/app.py'`; to run the tests, `'pytest tests/'` from the `module_5` directory.

2. Dependency Graph Summary

The dependency graph (`dependency.svg`, generated with `pydeps` and `Graphviz`) places `app.py` at the center of the application. `app.py` is the Flask presentation layer and imports two internal modules directly: `query_data` (which runs the SQL analysis) and `db` (the connection helper). `query_data` and `load_data` both depend on `db`, so `db` is the single chokepoint through which every database connection flows - a deliberate design that makes credential handling and connection logic easy to audit in one place. `pull_and_load` sits on the ETL side and depends on `load_data`, which in turn depends on `db`, forming a clean scrape -> save -> load -> database chain. Externally, the graph shows Flask (with its Jinja2 and Werkzeug sub-dependencies) on the web side and `psycopg` on the data side. Because every internal module funnels through `db` for data access, there are no circular dependencies and no module reaches the database without going through the shared helper.

3. SQL Injection Defenses

Every SQL query was refactored to use `psycopg`'s `sql` composition module instead of raw string building. No query uses f-strings, + concatenation, or `.format()` on SQL text. Each statement is constructed as a `sql.SQL(...)` object first, with table names wrapped in `sql.Identifier()` (which safely double-quotes them so they can never be parsed as SQL keywords or injection payloads) and constant LIMIT values wrapped in `sql.Literal()`. All user-facing and runtime values - term strings, statuses, GPA/GRE ranges, LIKE patterns - are passed as `%s` placeholders with a separate parameter tuple to `cursor.execute(stmt, params)`. This separation is the core defense: the query's shape is locked at construction time, and the database driver binds parameters as data only, so a malicious value such as `""; DROP TABLE applicants; -"` is treated as a literal string to match, never as executable SQL. Statement construction is always written as a distinct step before execution, making the separation explicit and auditable.

Module 5 - Software Assurance & Secure SQL

Ryan Gogerty - JHU Modern Concepts in Python, Module 5

4. Least-Privilege Database Configuration

Database credentials are no longer hard-coded. The app reads them from individual environment variables (DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD), assembled into a connection URL by `db._build_url()`; a single `DATABASE_URL` variable is still accepted as an override for CI. A `.env.example` file documents the variable names with placeholder values, and `.env` is git-ignored so real secrets are never committed. The application is read-only after the initial data load, so the database account it uses is granted only the minimum privileges required: `CONNECT` on the `gradcafe` database and `SELECT` on the `applicants` table. It is explicitly NOT a superuser and has no `INSERT`, `UPDATE`, `DELETE`, `DROP`, or `ALTER` rights. The SQL used to create it is: `CREATE USER gradcafe_app WITH PASSWORD '...'; GRANT CONNECT ON DATABASE gradcafe TO gradcafe_app; GRANT SELECT ON TABLE applicants TO gradcafe_app;`. This least-privilege posture means that even if the application's credentials leak, the blast radius is limited to reading public admissions data - an attacker cannot drop tables, alter the schema, or modify rows.

5. LIMIT Enforcement

Every query in `query_data.py` now includes an inherent `LIMIT` clause. Single-row aggregate queries (counts, averages, percentages) carry `LIMIT 1`. The two multi-row queries (Q10 top programs by acceptance rate, Q11 GPA by outcome) take a configurable limit parameter that is clamped to the range 1 to 100 by the line `'limit = max(1, min(int(limit), _MAX_LIMIT))'` before any query runs. This guarantees that no single request can ask the database to return an unbounded result set, capping the blast radius of a bug or an abusive oversized request and providing a predictable upper bound on memory and response size.

6. Continuous Integration Workflow

GitHub Actions runs a four-job pipeline (`.github/workflows/ci.yml`) on every push or pull request that touches `module_5/`. Job 1 (lint) runs `'pylint src/ -rcfile=.pylintrc -fail-under=10'` and fails the build if the score drops below 10/10. Job 2 (dependency-graph) installs `Graphviz`, regenerates `dependency.svg` with `pydeps`, and fails if the SVG is not produced. Job 3 (snyk) runs a `Snyk` supply-chain scan of `requirements.txt`. Job 4 (test) spins up a `postgres:17` service container and runs the full `pytest` suite with `'-cov-fail-under=100'`, failing on any test failure or coverage regression. This shift-left pipeline catches style, dependency, supply-chain, and correctness problems automatically before code is trusted. A local `Snyk` scan during development also found and fixed two real vulnerabilities - an `XXE` injection in `lxml 5.3.0` (High) and a cache exposure in `Flask 3.1.1` (Low) - by upgrading to `lxml 6.1.0` and `Flask 3.1.3`, after which `Snyk` reported no vulnerable paths across 61 dependencies.

7. Snyk Code (SAST) - Extra Credit

Module 5 - Software Assurance & Secure SQL

Ryan Gogerty - JHU Modern Concepts in Python, Module 5

In addition to the dependency scan, Snyk Code static application security testing was run with 'snyk code test src/'. The initial scan surfaced two MEDIUM findings in first-party code. First, an Open Redirect in templates/base.html: the update-analysis handler assigned the fetch response URL directly to window.location, which a crafted redirect response could abuse; this was fixed by navigating to the fixed, trusted internal path '/analysis' instead of echoing the response URL. Second, Debug Mode Enabled in app.py: the development server called run(debug=True), which exposes the Werkzeug debugger if reachable by untrusted parties; this was fixed by reading the debug flag from the FLASK_DEBUG environment variable and defaulting it to off, so the application is secure by default and debug must be explicitly opted into for local development. After both remediations, a re-scan reported zero issues. Output evidence is saved in snyk-code-analysis.txt.